Beyond Recall: Recovery Contracts for Agent Memory Mutations

Aditya Mantri*
Independent Researcher

October 29, 2026

Abstract

Recovering a memory's expected facts need not preserve its physical records, current authority, or retrieval context. We investigate these distinctions through native public implementations and an independently controlled recovery experiment. In a pinned Mem0 ingestion path, exact-hash suppression uses only ten retrieved candidates. If extraction re-emits N stored literal facts, this admits at least $\max(0, N - 10)$ copies under stated conditions. Ninety-six controlled native-pipeline executions confirm the boundary. A separate live experiment makes 28 Sol 6.1 requests, induces four actual worker exits after committed writes, and compares 48 fresh-process retries. Fresh-model native retries add 91 literal copies and one ungraded formulation; two targeted interventions add none. In a post hoc 1,008-query exposure test, every exact target remains present, yet the 40-fact native retry state returns only 247 summed per-query distinct strings in 400 retrieval slots, versus 400 in baseline. Additional probes identify bounded MemTX snapshot-description discrepancies and distinguish SQLite rollback from a grouped Redis error path. The original 528-case factorial separately varies stable addresses, retained decisions and completion receipts. The contribution is a conditional admission mechanism, measured context redundancy, and auditable contract witnesses. The recovery primitives and broader evaluation beyond recall are established; no semantic-quality gain, production incidence, or new transaction algorithm is claimed.

1 Introduction

An agent decides to remember three preferences. Its first write commits, but the process stops before the remaining writes finish. On retry, should it choose a new plan, replay the previous plan, or return a recorded outcome? A fact-set check cannot fully answer this question. Repeating an insertion may duplicate a physical record without changing the fact set. Reusing an address may suppress that duplicate while writing a different value. Replaying the original value may restore the expected fact while destroying a newer update.

These questions matter because memory systems update and forget information across interactions [9, 31, 52]. They are also grounded in established systems research. RPC failure can leave execution uncertain; RIFL ties completion recognition to durable effects; Beldi replays logged choices [4, 18, 50]. Recent agent work evaluates ambiguous effects and recovery evidence across models and persistence hosts [19, 25]. Evaluation beyond recall is likewise established in governed-memory and lifecycle benchmarks [3, 45, 53].

Our principal empirical question is whether a bounded retrieval window is also being used as a write-admission authority. In the tested Mem0 path, the same ten candidates supply both extraction context and the exact-hash suppression set. Re-emitting a larger stored batch can create exact physical copies. Those copies can subsequently occupy a fixed retrieval budget while a target-presence check continues to pass. Section 3 derives the conditional boundary, isolates it in controlled native calls, tests live extraction after process failure, and measures its retrieval exposure.

A complementary question concerns recovery obligations themselves: how do stable effect addresses (D), retained decisions and targets (P), and terminal receipts (R) affect content, physical cardinality, acknowledgement and work? The original factorial (Sections 4–6) uses a controlled decision tape to isolate those observations. Further extensions examine native checkpoints, recorded plan compaction, competing writers, public snapshot semantics

*Independent research. Author background: AI Engineer at Genies; M.S. in Engineering, Purdue University.

and backend error boundaries. These layers have separate protocols and denominators; they do not form a pooled reliability score.

The contribution is a concrete admission-to-retrieval mechanism with bounded public-implementation evidence, supported by an executable decomposition of recovery contracts. This is not a claim that deduplication, durable replay, lifecycle evaluation or transaction validation is new. The artifacts contain synthetic inputs, source, databases, process witnesses, raw model outputs and separately implemented checkers. No proprietary application implementation or nonpublic conversation data is used.

2 Related Work

Memory and lifecycle evaluation. MemGPT, MemoryBank, and Mem0 develop persistent conversational memory, while A-MEM, Zep, Hindsight, and MemOS explore linked, temporal, retention-oriented, and operating-system abstractions [9, 17, 21, 31, 37, 48, 52]. LoCoMo and LongMemEval evaluate long-term conversational use; MemoryAgentBench evaluates incremental interactions; HaluMem studies hallucinations; UTILMEM studies utilization of distributed evidence [6, 13, 23, 36, 47]. Those semantic capabilities are not measured by equality of synthetic fact strings.

TimlMemBench separates canonical-pool answering from ingestion and governed-use cases such as revocation, supersession, scope, and provenance [53]. Prospective-memory evaluation distinguishes standing constraints and release authorities [3]. Memora evaluates consolidation and mutation, including obsolete information in responses [45]. Our execution-level observations complement these evaluations; we do not introduce evaluation beyond recall or show improved semantic memory quality. AgentPoison and MINJA address malicious memory contents under different attacker assumptions [7, 10]. Faithfully replaying a poisoned choice would not solve either threat.

GEM/MemState frames memory as a governed state trajectory, TOKI specifies bitemporal operators and retained judge decisions, and MemTX adds staged validation, action gating and cascading repair [20, 30, 46]. Revocation enforcement and execution-state unlearning distinguish hidden records from downstream influence [40, 49]. We do not reproduce their semantic benchmarks. In particular, the Mem0 exposure arm in the revocation study explicitly includes expired records; it should not be generalized to default expiry filtering.

Public Mem0 discussion and source-reading notes already identify top-ten candidates as a limit on semantic duplicate recognition [14, 32]. Our narrower investigation checks literal hash protection when that same window supplies the suppression set. The conditional $N - k$ witness and downstream exact-string exposure should be read as a refinement of this known boundary, not a first discovery of duplicates in agent memory or a proof that no exact antecedent exists.

Recovery and effect contracts. RIFL combines request identity, durable completion results, retry rendezvous, and safe reclamation; completion metadata must be atomic with its effects [18]. Beldi retains nondeterministic results and supports logged replay under its storage/API assumptions [50]. ARIES addresses database recovery, and sagas address application compensation for long activities [12, 28]. None of those guarantees follows merely from writing an agent plan to one file and its effects to another.

SagaLLM applies persistent context, validation, and compensation to agent planning. Atomix separates execution and settlement, while Agentic Transaction considers semantic transaction properties, including memory [5, 27, 44]. ACRFence evaluates restoration/replay and authority risks, distinguishing its implemented evaluation from a proposed analyzer [51]. LIMBO grades uncertain effects against service-side ledgers across models and harnesses; Permission to Retry separates recovery evidence, authority, completed work, and unresolved obligations on native hosts [19, 25]. These are close antecedents, not defeated baselines. Mansoor et al. combine postcondition verification, verify-before-retry, and action/payload-derived keys in a LangGraph agent evaluated on simulated non-atomic failures [24].

Agent-concurrency work examines long read–generate–write histories and snapshot insufficiency; Cordon composes tool flows through lineage, shadow state and an effect outbox [8, 15]. Root-scoped quiescence combines epochs, protected-effect fencing and immutable bindings [54]. These are direct antecedents to the mechanisms used in our controls.

Systematic fault selection also predates this study: lineage-driven fault injection searches failures within a bounded model [1]. Our primary histories are manually specified rather than an exhaustive fault search. The contribution is

the independently controlled D/P/R decomposition, inspectable observations, and boundary witnesses. Section 7 adds native persistence and model-output evidence while preserving that distinction.

3 From Write Admission to Retrieval Exposure

The principal public-implementation experiment asks whether a bounded set of retrieved memories can safely stand in for the complete store when admitting repeated literal facts. We test the synchronous inferred-ingestion path of Mem0 at commit `26cdefb4`, whose package version is 2.2.1 [26]. The source retrieves at most ten write-time candidates, constructs its existing MD5-hash set from those candidates, suppresses hashes present in that set or already emitted within the current batch, and assigns fresh UUIDs to admitted records. This is an implementation observation, not a vendor promise of exactly-once ingestion.

3.1 A conditional admission boundary

Let S contain N distinct literal hashes already stored for the selected user, V the retrieved candidates with $|V| \leq k$, and X the distinct literal hashes emitted by extraction. Ignoring hash collisions, the tested admission rule suppresses $H(V)$ and within-batch repeats. The hashes of already stored facts that it may admit are therefore

$$B = (X \cap S) \setminus H(V). \tag{1}$$

If extraction re-emits all of S, or at least every member of $S \setminus H(V)$, then

$$|B| \geq \max(0, N - k). \tag{2}$$

This is an elementary conditional counting argument. It is neither a new deduplication theorem nor a prediction that an arbitrary LLM will emit those facts. It identifies a precise obligation: a semantic candidate window does not establish global literal absence. Persisting a nondeterministic extraction result also does not make replay of its physical effects idempotent when admission assigns fresh addresses.

3.2 Controlled native-pipeline experiment

Before measured execution, we froze eight batch sizes $(1, 5, 10, 11, 13, 20, 21, 40)$, two explicit extraction controls, two embedding implementations, and three arms: $8 \times 2 \times 2 \times 3 = 96$ parameterized executions. Each execution invokes native `Memory.add` initially and twice more with the identical source, giving 288 calls. One extractor re-emits all literal facts; the other emits only facts absent from the visible candidate texts. Both are controlled callbacks, not measured LLM behavior, and deliberately do not use past messages for deduplication.

The embedding factor compares a nonsemantic hash control with actual local BGE-small-en-v1.5 inference (384 dimensions), through FastEmbed 0.9.1 and Qdrant-client 1.20.0 local mode [2, 34, 35]. The native arm retains $k = 10$. An authored search wrapper widens that write-time request to 10,000; a separate authored sequential filter checks complete current-store exact text before returning extracted facts. These interventions are not stock Mem0 flags or atomic concurrent-admission fixes. Native storage and admission source remain unchanged. Telemetry is disabled; model files are prewarmed and measured controlled executions disallow external connections.

All 96 executions preserve the expected final literal fact set. For every native configuration with $N > 10$, physical record counts are N, $2N - 10$, and $3N - 20$ across the three calls; the 40-fact case yields 40, 70, and 100 records (Figure 1). All sizes at or below ten and both intervention arms admit no extra copies. Twenty of 32 native parameter cells have extra physical records. Repetition across controlled extractors and embedders establishes this finite mechanism under those configurations, not independent failure types or a population incidence estimate. A separate checker verifies every raw Qdrant/history store, admitted response, candidate set, and frozen source/specification identity.

3.3 Live extraction and actual process recovery

A separate frozen protocol uses four authored source batches ($N = 8, 12, 24, 40$) and the native role-designated extraction prompts with requested Sol 6.1 through a personal Codex account [29]. Each baseline call commits

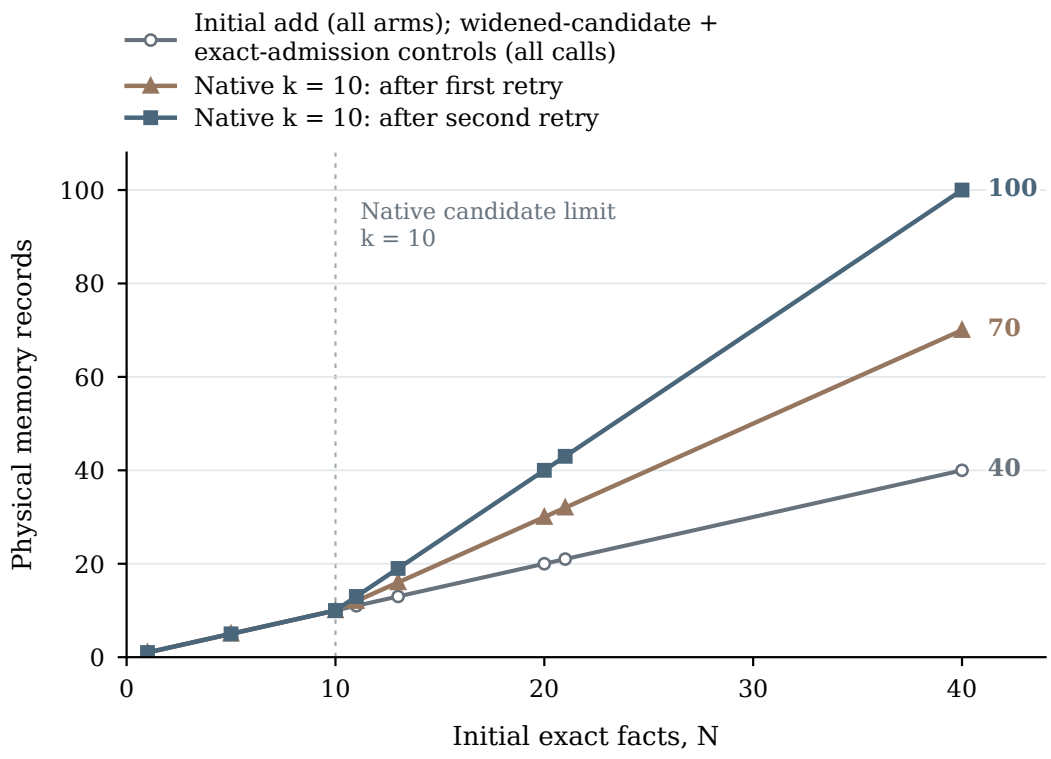


Figure 1: Controlled native Mem0 admission with real BGE embeddings and the visible-candidate extraction control. Initial calls and both intervention arms retain N records; native exact-source redeliveries exceed N above the ten-candidate boundary. The other three extractor/embedding combinations agree. These are parameterized controlled executions, not live-model failure-rate estimates.

native effects, returns from `Memory.add`, records an evaluator witness, and then actually exits its worker with `os._exit(87)` before logical acknowledgement. All four baselines extract and store exactly N distinct facts and reach this exit. These are client/process cuts after effects, not network packet loss or storage mid-commit failure.

Fresh processes reopen copies of each baseline's native Qdrant/history state, preserving original record identities. Four batch sizes, two recent-message settings, three arms, and two extraction-recovery strategies give 48 retries. The fresh strategy makes 24 new model requests; the recorded-result strategy reuses the baseline response exactly and makes none. Together with four baseline requests, there are 28 actual requests. Every retry completes normally.

The recent-message settings retain native history or append twelve neutral messages so that the native last-ten-message window no longer contains the original source. Even the fresh-history prompt truncates each past message to 300 characters; the repeated new-source message remains complete. Native Summary and Recently Extracted Memories remain their default empty values in both settings; we do not disable a populated cache. These are explicit prompt/cache preconditions. They are not independently sampled conversational histories.

For each context setting, fresh-model native retries add $0, 2, 14, 30$ records across the four sizes. In fresh history all 46 additions are exact baseline-text copies. In aged history 45 are exact copies and one is a new literal formulation, which we leave semantically ungraded. Thus the eight fresh-model native retries add 91 exact copies and one new string. Recorded-result native retries add 92 exact copies. Both interventions add zero records in all corresponding retries. Original identities and values survive in every arm. The one new formulation prevents

treating Equation 2 as a universal literal-count equality for fresh model output.

The evidence includes full native prompts, raw and used responses, candidate traces, request statuses and 52 raw baseline/retry stores. Configuration injection changes the extraction provider, not native admission source. The requested alias, CLI version and visible usage are retained; the exact backend model snapshot, harness system prompt, temperature and seed are not exposed or controlled. This is one model/harness and four source batches, not a model comparison, semantic error rate or natural-workload estimate.

3.4 Target presence can coexist with redundant context

After observing the live writes, we froze a post hoc exposure protocol before issuing its measured queries. It uses every distinct baseline fact as a literal query (84 across the four sizes), four store states (baseline and each fresh-history/fresh-model retry arm), and retrieval budgets $K \in \{5, 10, 20\}$: $84 \times 4 \times 3 = 1008$ native `Memory.search` calls. All states use the same reader, BGE/BM25 configuration, zero score threshold, and `show_expired=False`. Optional spaCy is absent and the native fallback is used. The write-time candidate bound $k = 10$ and reader budget K are different parameters.

The fresh-history states hold literal content and original identities fixed; every added point is an exact baseline-text copy, with its BGE vector matching within 10^{-6}. This deliberate selection excludes the aged retry's ungraded formulation. It is a mechanism probe, not held-out generalization. Define per-query returned slots r_q, distinct literal strings u_q, duplicate slots $d_q = r_q - u_q$, and exact target presence t_q. All 1008 queries have $t_q = 1$. Nevertheless, at $N = 40, K = 10$, native retries return 400 slots with only 247 summed per-query distinct strings, versus 400 distinct strings in baseline and each intervention. The 153 duplicate slots reduce mean distinct strings per query from 10 to 6.175 (minimum 5); Figure 2 shows all four sizes.

Across all budgets, 213 query cells contain duplicate slots, but only 201 have fewer distinct strings than matched baseline. For $N = 12, K = 20$, duplicate records occupy previously spare capacity without reducing the twelve distinct strings. Thus duplication and displaced evidence must be checked separately. No answer-generation task, downstream harm, semantic relevance annotation, latency or token-cost measurement is performed.

A separately implemented checker reconstructs coverage, raw identities/vectors/history, query results and paired metrics. The original runner deletes its temporary reader copies, so a frozen retention wrapper repeats the same computation and retains all sixteen after-close stores. All 1008 ordered result-identity lists and metrics match; retained point blobs/history are unchanged. This is computational reproduction by the same study, not 1008 additional independent samples. The empirical contribution is the conjunction of a bounded admission witness, native live recovery evidence, and measured context redundancy despite perfect exact-target presence.

4 Contract and Factorial Design

4.1 Intent, source, and decision

A logical intent j identifies one update; attempts a retry it. Every primary case has isolated stores and one fixed intent. The shared frontend selects enabled conversation rows with generation g and turn at most endpoint e, ordered by turn. The fixture uses $g = 0, e = 1$. Future and disabled rows make the source controls discriminating.

The controlled planner retains the latest selected fact per attribute and chooses an ordered operation list

$$P_j^{(a)} = \Pi(H_{e,g}, \rho, \xi_a) = (p_0, \ldots, p_{m-1}), \qquad p_i = (\mathrm{kind}_i, \mathrm{fact}_i, \mathrm{target}_i). \tag{3}$$

Here ρ is an operation profile and ξ_a a tape setting. Puts name an existing target or request insertion; deletes name a target. Changed-tape histories prescribe `topic=running` instead of `topic=cycling`. This is a selected continuity adversary, not a measured model error or natural drift rate. The primary planner neither calls an LLM nor infers targets from the evolving store.

4.2 Three controls

D: stable addresses. Existing named targets are bound under either setting. For targetless puts, D1 uses the first 24 hexadecimal characters of SHA-256 over intent and instruction slot (j, i); D0 generates a fresh 12-byte

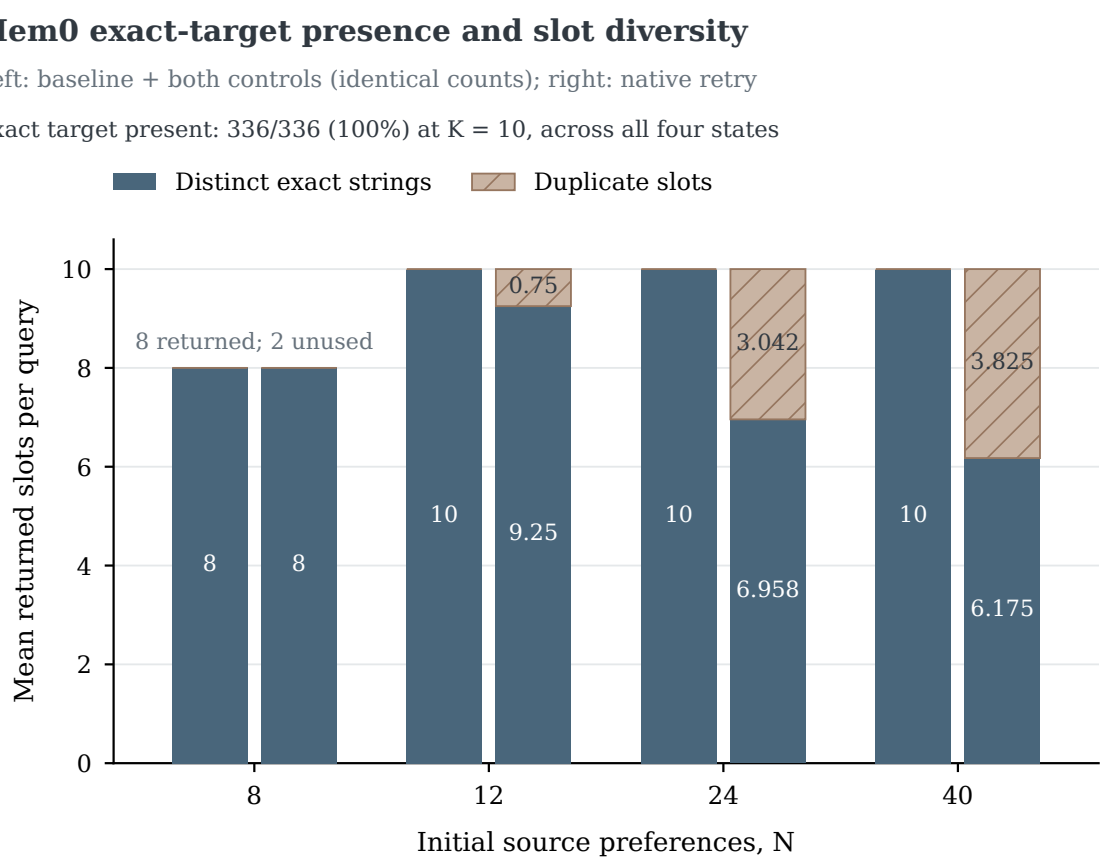


Figure 2: Retrieval exposure at budget ten across all four authored batch sizes. Bars separate distinct exact strings from duplicate slots, averaged over every baseline-fact query. Both intervention states match baseline. Every exact target is present. Distinct literal strings are not semantic diversity or answer quality; unused capacity for batches smaller than the budget is not duplication.

random address at each application. D controls where a put writes, not which value it writes. Collision probability and identity allocation are outside this finite experiment.

P: retained decisions. P1 commits the literal ordered operations and target bindings before effects, then reloads that list on retry. P0 chooses again on reaching planning. Targetless operations remain targetless in the retained list: D0 generates fresh addresses even when P1 preserves the decision.

R: completion receipts. R1 checks a terminal receipt after the entry generation check, returning a recorded outcome on a hit. It stores the receipt after effect processing. R0 omits that lookup and storage. A P1 decision remains retained after success under *both* R settings. This fixed retention rule isolates receipt work suppression.

4.3 Separate observations

Let $\mathcal{R}$ be terminal physical records, $\phi(r)$ the fact in record r, E the expected fact set, and A a normal final process exit with an acknowledgement event. Define

$$C \equiv \{\phi(r) : r \in \mathcal{R}\} = E, \qquad K \equiv |\mathcal{R}| = |E|, \qquad T \equiv C \wedge K \wedge A. \tag{4}$$

C collapses identical duplicate facts; K observes physical row count. The selected terminal check T does not incorporate decision continuity. If $\mathcal{D}_j$ contains digests of decisions reported as chosen or loaded for intent j,

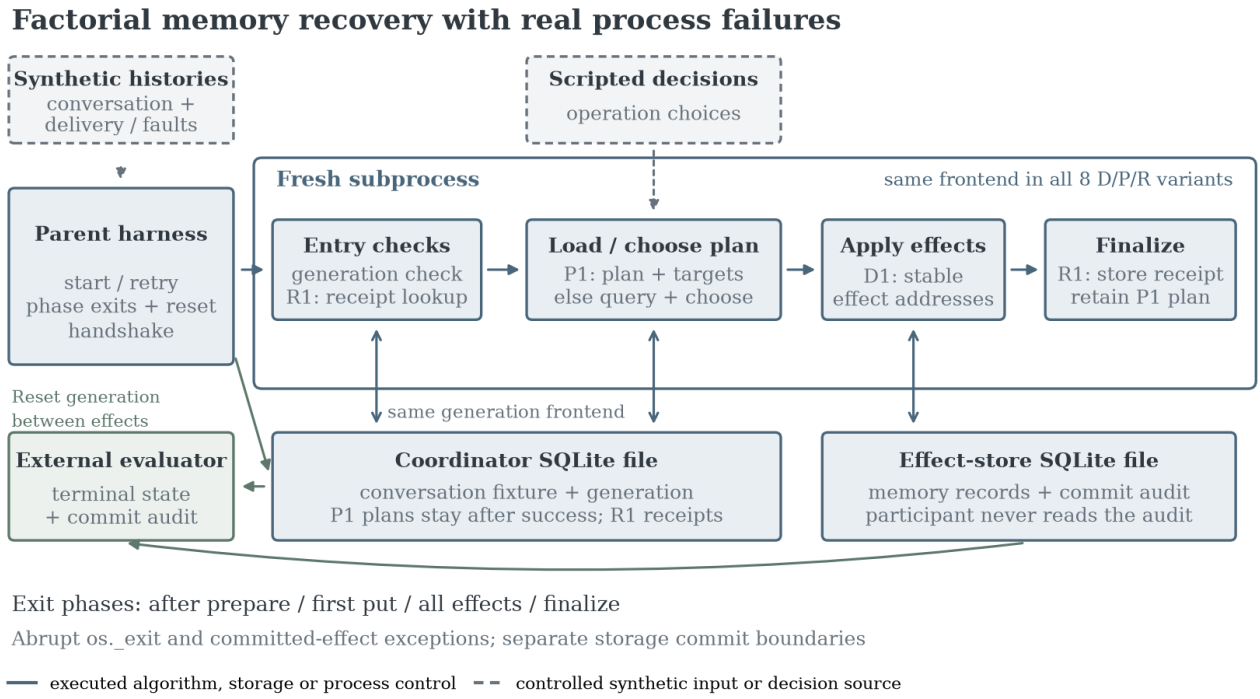


Figure 3: Primary factorial architecture. Controlled source, decision tape, and fault histories feed a common frontend and eight D/P/R policies. Fresh subprocesses reopen separate coordinator and effect stores. Generation checks precede receipt lookup and follow the effect loop; receipt/plan hits bypass source selection. The effect audit is evaluator-visible and not read by the participant. Dashed boxes are controlled inputs. The process frame does not imply cross-store atomicity.

continuity is

$$L \equiv \bigwedge_j (|\mathcal{D}_j| \leq 1). \tag{5}$$

A reported choice before durable preparation still counts for L. Planning invocations Q count decision-chosen events. Committed work W counts one audit row for each participant put/delete commit, including repeated deletion of an absent target. It excludes reset/cleanup SQL, reads, coordinator writes, tokens, latency, and monetary cost.

5 Implementation and Failure Model

The primary implementation uses Python's standard library and two SQLite files. The coordinator stores conversation, generation, decisions, and optional receipts. The effect store holds addressed records tagged by intent/generation, plus a commit audit. Each put/delete and audit entry commit together. The participant never reads the audit for deduplication. Preparation, effects, receipt finalization, reset, and cleanup have separate commits; no transaction spans the files. SQLite uses `synchronous=FULL` and `journal_mode=DELETE` [42].

Insert performs three targetless puts. Revise includes an existing-target unit update and two insertions. Retire deletes an obsolete target before three insertions. Mixed performs an existing-target unit put, a tone insertion, the named deletion, and a topic insertion. An exit after the first put need not be after the first effect.

Each fresh attempt executes the following order:

1. Compare current and expected generation. On mismatch, delete this intent's old-generation records, acknowledge cancellation, and return.
2. Under R1, return the recorded outcome if a matching receipt exists.

Table 1: Primary outcomes, 66 specified cases per policy. C, K, T, L count passes; Q, W count planning invocations and committed put/delete calls. Every policy acknowledges all 66 final attempts.

Policy	$C\uparrow$	$K\uparrow$	$T\uparrow$	$L\uparrow$	$Q\downarrow$	$W\downarrow$
D0P0R0	42	37	29	42	112	317
D0P0R1	50	49	41	50	100	275
D0P1R0	66	37	37	66	66	317
D0P1R1	66	49	49	66	66	275
D1P0R0	42	66	42	42	112	317
D1P0R1	50	66	50	50	100	275
D1P1R0	66	66	66	66	66	317
D1P1R1	66	66	66	66	66	275

3. Under P1, load a retained decision; otherwise query eligible source and choose a list, saving it under P1. This is the preparation boundary.
4. Apply operations in order, committing each effect and audit row.
5. Recheck generation after the loop. On mismatch clean up old-generation intent records and select cancelled; otherwise select applied.
6. Under R1, commit the outcome receipt. Keep any P1 decision. Acknowledge and exit.

The harness terminates the child with `os._exit(87)` after preparation, the first put, all effects, or finalization, then retries in a new process [33]. Barriers follow returned commits; no SQLite commit is interrupted. Controlled unavailable-response failures raise an exception after a put/delete commit and exit with code 74.

Reset-before-retry advances generation after a first-put exit and removes fixture-origin rows, leaving intent residue for entry cleanup. Reset-between-effects uses a parent/child handshake: the child pauses after the first put, the parent advances generation and clears effects, then the child resumes. The post-loop check removes subsequent stale writes. This tests terminal cleanup, not absence of transient stale reads.

6 Primary Results

6.1 Coverage and evidence

Seventeen histories, four profiles, and eight policies yield $17 \cdot 4 - 2 = 66$ cases per policy and 528 total: lost-delete-reply excludes two profiles without deletes. Histories include normal execution, fixed/changed completed redelivery, four exit barriers, unavailable put/delete replies, source controls, and two resets. Every case has distinct stores; renamed entities or reruns do not enlarge coverage.

Source and specification were frozen after development, before the final campaign. They were not externally preregistered or held out. Python 3.12.13/SQLite 3.53.1 produced 928 fresh attempts: 592 normal, 256 abrupt, and 80 controlled-exception exits. Abrupt exits comprise 64 after preparation, 96 after the first put, 64 after effects, and 32 after finalization. All final attempts acknowledge, so A does not distinguish main outcomes.

6.2 State, decision, and work

Table 1 gives the complete factorial. D0P0R0 passes content in 42 cases but T in only 29. P1 with D0R0 raises content to 66 while leaving 29 cardinality failures. D1P0R0 instead passes cardinality in all 66 but retains 24 content failures.

A first-put insertion exit illustrates the separation. The first attempt inserts `tone=concise`. D0P1R0 replays three facts to fresh addresses: content remains correct, but four rows exist. D1P0R0 with a changed tape reuses three addresses yet writes `topic=running`: cardinality passes while original-intent content and continuity fail. D1P1R0 preserves both by retaining the original tape and addresses.

Switching D adds 13, 9, 29, and 17 terminal passes in $(P, R) = (0,0), (0,1), (1,0), (1,1)$ order, without changing content, continuity, planning, or work counts. Switching P adds 24 content/continuity passes under R0 and 16 under R1, saving 46 or 34 planning calls respectively without reducing effect calls. Its terminal gains are 8, 8, 24,

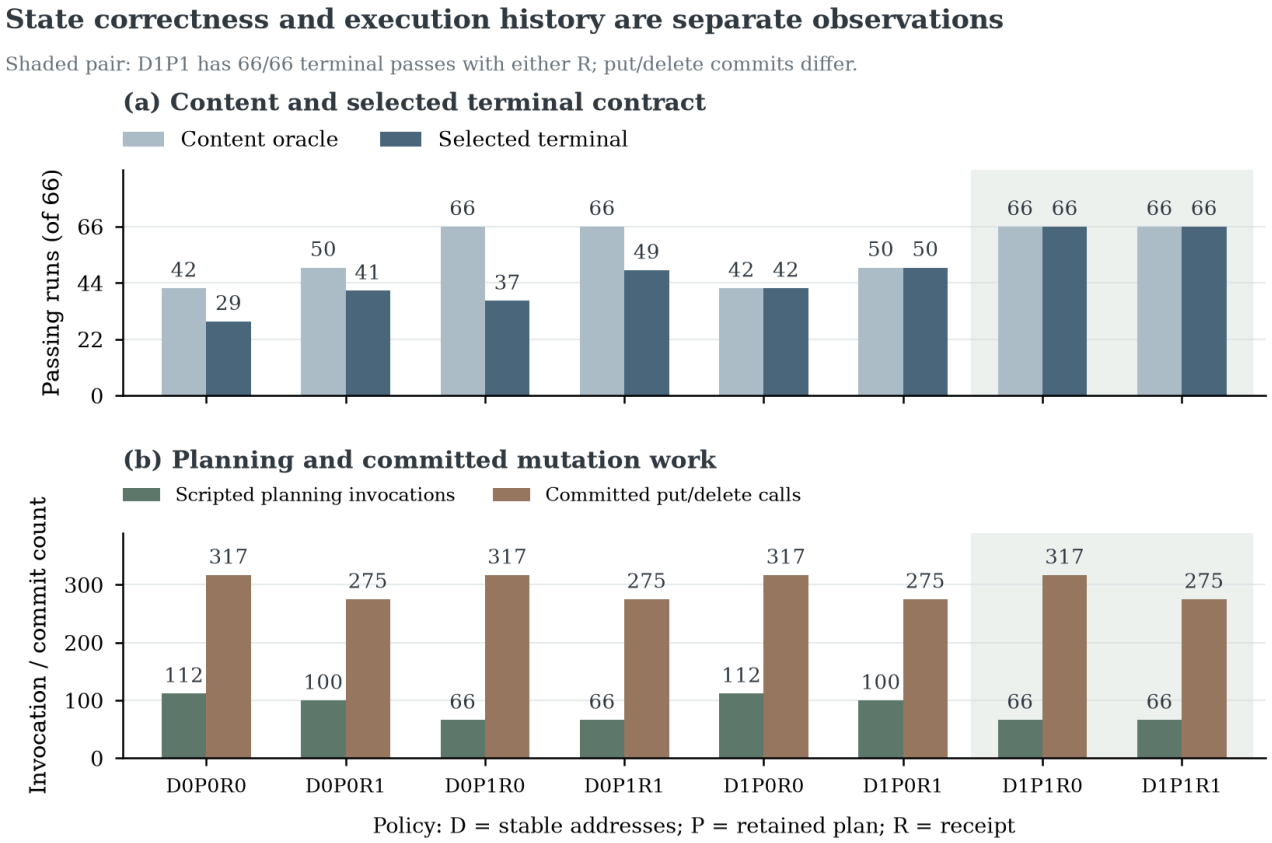


Figure 4: Primary content and terminal outcomes, with planning and committed-effect counts. Each policy has 66 structural cases. In the shaded D1P1 pair, both receipt settings pass T, but R1 suppresses 42 committed put/delete calls. Counts describe specified schedules, not population reliability or generic cost.

Table 2: Separate D1P1R1 mixed-profile probes. All final attempts acknowledge. Q and W are counts, not pass metrics.

Probe	C	K	T	L	Q	W
Remove receipt only	pass	pass	pass	pass	1	8
Remove decision only	pass	pass	pass	pass	1	4
Remove both	fail	pass	fail	fail	2	8
Exit before decision store	fail	pass	fail	fail	2	4
Intervening external write	pass	pass	pass	pass	1	5

and 16 in (D, R) order.

Switching R adds 12, 12, 8, and 0 terminal passes in (D, P) order. D1P1 passes all 66 T and L checks and plans 66 times under either R setting, but W falls from 317 to 275. A completed mixed-profile redelivery explains four saved calls: R0 replays three puts and a delete, whereas R1 returns the receipt. Receipt suppression and final-state conformance are distinct benefits.

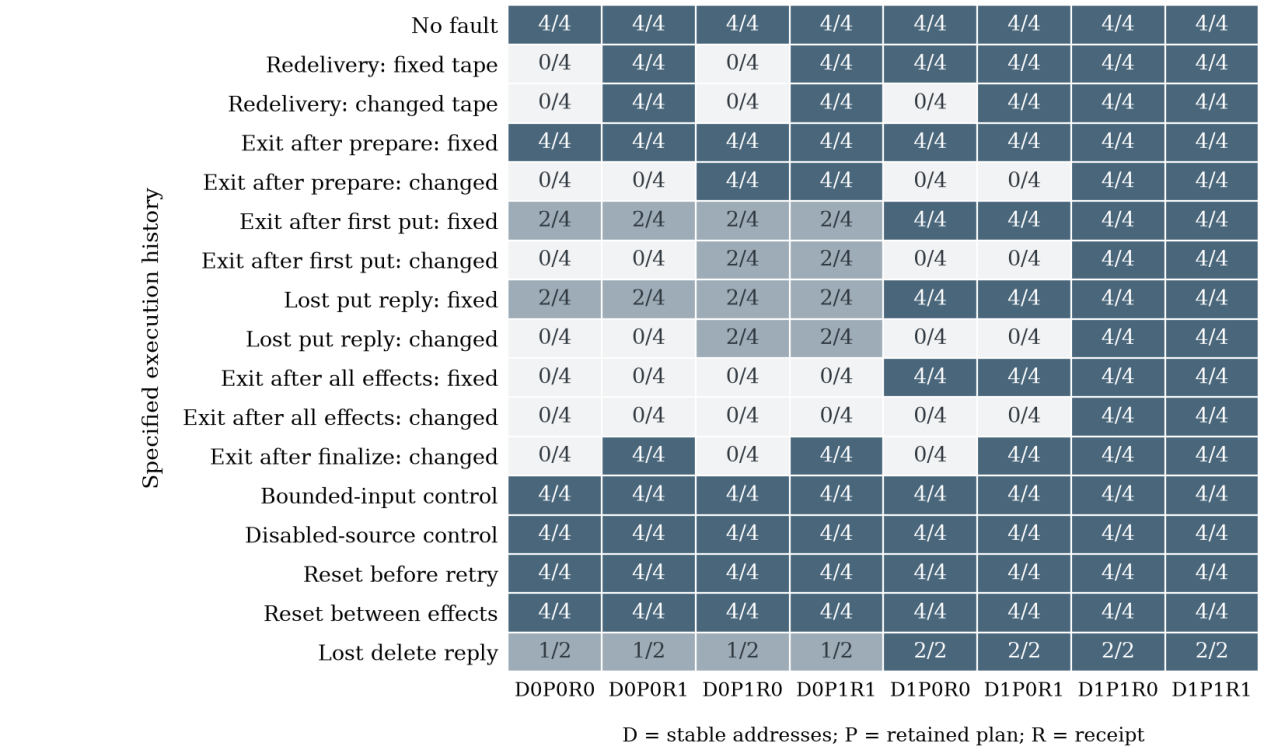


Figure 5: Terminal passes by declared history and policy. Cells show passing profiles/applicable profiles; lost-delete-reply has two profiles, all other histories four. Changed-tape cases prescribe a different later decision. Source and generation controls are shared, so their successes do not estimate a D/P/R gain.

6.3 Five boundary probes

Deleting only the receipt repeats the retained decision; deleting only the decision leaves the completed receipt fast path. Removing both permits a changed-tape replay. These are explicit deletion tests, not expiration policies. The before-store exit loses a reported but uncommitted choice, outside the prepared-decision obligation.

The last probe inserts a newer external value after the first put and before retry. Replaying the retained decision overwrites it with `unit=metric`. All original-intent checks pass despite losing that newer value. Thus T cannot certify latest-writer preservation; Section 7.4 tests that distinction under stronger store contracts.

6.4 Checking and reproduction

A separate checker reads SQLite state and process witnesses without importing participant metric functions. It recomputes all observations and structural keys. A protocol audit checks targets, retained metadata, generation, and reconstructed state, including 688 primary source-selection observations. All 528 cases and five probes pass evidence-consistency checks. Ten focused tests cover actual exit/resume, oracle separation, fixed retention, source/generation controls, repeated deletes, changed decisions, and rejection of forged metrics.

A fresh same-host campaign with Python 3.14.3/SQLite 3.51.2 matches every primary scalar outcome and all five probes with unchanged source. Completed Ubuntu Python 3.12/3.14 runs also match frozen aggregates after raw-state verification. These are software-environment reproductions by the same study, not third-party replication

or additional primary cases.

7 Reference and Boundary Extensions

7.1 Native checkpoint reference

A separate implementation uses LangGraph 1.2.18, checkpoint 4.3.1, and the SQLite checkpointer 3.1.2 [16]. A genuine `StateGraph` runs choose-decision → apply-effects → end with synchronous durability. Plans and completed results reside in native checkpoints; the primary coordinator's decision/receipt tables remain empty. On redelivery, application code inspects `graph.get_state`; an empty next-node list with a completed outcome takes an explicit return path without invoking the graph again. Application source/generation checks and the external effect ledger retain the same semantics.

All 66 primary D1P1R1 cells have corresponding native barriers. Preparation exits occur at apply-node entry after the decision checkpoint; prefix and post-effect faults occur inside that node before return. Finalization exits follow a returned completed invocation. Every attempt starts a fresh process and resumes from native state.

The reference matches all seven observations in every cell: C, K, T, L, A each pass 66, $Q = 66$, and $W = 275$. Its 116 attempts include 32 abrupt exits. A checker inspects 260 actual checkpoint rows, retained plans/results, effect state, and barrier witnesses. Nine focused tests pass. The single effects node deliberately matches the primary plan/apply decomposition; per-operation native tasks could change W and were not evaluated. This is portability evidence, not framework optimality or a comparison against a deployed memory system.

7.2 Recorded Sol 6.1 planning pilot

We froze a small protocol before requesting 24 fresh Codex sessions with `gpt-6.1-sol`, medium reasoning requested. Six synthetic conversations cover insertion, correction, retirement, mixed edits, a move, and a conditional preference. Each receives two repeats on the original store and two on a store after the first operation from original repeat 0. Both partial repeats see the same snapshot. Requests use an empty working directory, ephemeral session, ignored user configuration, disabled project documents, read-only permissions, and a fixed JSON schema [29].

All 24 responses satisfy the operation schema. Seven of 12 identical-prompt pairs differ in literal arguments even after ignoring JSON key order and operation-list order; examples include `reply_style` versus `reply_length` and synonymous wording. All 12 original-to-partial comparisons differ. The latter compare changed inputs, not same-input variability. A separate checker verifies prompts, raw responses, the partial-snapshot rule, and counts.

Literal variation does not establish semantic disagreement or harmful drift. The pilot has one requested alias, one harness, six authored conversations, and two repeats per condition. Exact backend snapshot, temperature, seed, and system prompt are not controlled or exposed by the retained CLI events. We report observations, not a drift-rate estimate, accuracy score, or statistical significance test.

7.3 Plan compaction and positional identity

After examining the recorded outputs, we froze an exploratory integration protocol: 12 original-repeat-0/partial-repeat plan pairs, each replayed in three SQLite arms. All arms start from the exact archived partial snapshot with a common bijective public-ID/physical-address binding. The arms use (i) the recorded partial plan with stable intent/slot addresses, (ii) that same plan with fresh insertion addresses, or (iii) the retained original plan with its original stable addresses. Named targets are held fixed.

Two positional-replan integrations overwrite a completed prefix record with a different fact key; the other ten preserve the prefix. Fresh-address and retained-plan controls preserve it in all 12 pairs each. Both overwrites are the two partial responses of the same preferences scenario, the only scenario with a targetless first operation. They are not two independently sampled failure types or an estimated model failure rate.

The witness requires no contradictory model choice. Original slot 0 stores `reply_style`; the partial snapshot already contains it. The model omits that completed fact and begins its remaining plan with `preferred_units`. The adapter derives the same physical slot-0 address and overwrites the style record. Giving the identical remaining plan fresh addresses prevents that overwrite; retaining the original ordered plan preserves the original

identity-to-operation binding. All 36 integrations, their before/after audit rows, and three focused control tests are retained. No additional model requests or semantic-synonym grading are involved.

This is an adapter contract failure, not evidence of a LangGraph defect or unsafe idempotency keys in general. LangGraph already warns that changing task/interrupt order can mismatch cached values [16]. Featonby describes retaining original parameters and rejecting changed intent under the same request identifier [11]; our positional upsert has no such rejection. The concrete model-output witness illustrates that known identity/order hazard when a memory planner compacts a plan by omitting an already represented fact. Stable logical-operation identities or mismatch rejection are additional controls, not evaluated remedies here.

7.4 Conflict recognition and stale overwrites

An exploratory store-contract extension compares unconditional upsert, value comparison, monotonic-version comparison, and version comparison with an atomic per-operation receipt. A successful mutation and its receipt commit in the *same* SQLite transaction in the last contract, unlike the primary cross-store terminal receipt. In that receipt-enabled contract, once a successful operation has a retained receipt, reusing its identity with changed arguments is rejected; a failed comparison does not bind that identity. Deletion retains a versioned tombstone.

The bounded campaign contains 48 sequential schedules (six first-before-retry linearizations of two intents, two put/delete profiles, four contracts), 20 named witnesses, and 16 two-process races. Both processes rendezvous before writing; SQLite serializes transactions, and the winner is uncontrolled. Reply-loss witnesses model reissued calls after a commit; no transport packet is dropped. The checker reconstructs state from the raw effect journal without importing store code. All 84 cases pass declared evidence checks; eight focused contract tests pass.

Three witnesses identify different obligations. First, an exact replay after its own committed write receives a version conflict under version comparison alone, but historical-success acknowledgement under the per-effect receipt. Second, after A commits, B commits a newer value, and A retries, unconditional upsert restores A and passes its original-intent content check while erasing B. The versioned receipt recognizes A's historical effect without changing B. Third, value-only comparison accepts a stale operation after a value changes and returns to its old value (ABA); monotonic versions reject it. The delete/recreate witness retains this distinction through a tombstone.

These apply established idempotency and concurrency ideas, including RIFL's distinction between executing a request once and preserving its result against subsequent writers [18]. They are not a new transaction protocol. For a single-valued target, if A's desired value is x and a later accepted writer establishes $y \neq x$, then

$$C_A \Rightarrow v = x, \qquad F_B \Rightarrow v = y, \qquad C_A \wedge F_B = \text{false}. \tag{6}$$

Here F_B means preserving B absent an accepted later update. Historical acknowledgement of A can be valid while C_A is false. Evaluation must specify which obligation takes precedence.

Under retained immutable receipts and monotonic versions, the narrow per-effect invariant follows directly: an exact successful replay finds a receipt and does not mutate; an unseen stale operation fails its version check. Neither branch overwrites a newer version. The argument assumes atomic receipt/effect storage, fixed identity/arguments, and retained version history. It proves neither whole-plan atomicity nor safe reclamation and does not transfer to the primary two-file receipt scheme.

8 Public Contract and Backend Boundaries

The following extensions test different contracts; their cells are not pooled with the admission experiment or primary factorial. They check bounded public behavior rather than certify an industry standard.

8.1 MemTX snapshot descriptions and public implementation

MemTX's v2 paper describes start-time visible-record snapshots, snapshot reads, and rejection of same-slot rivals committed after a transaction's snapshot before authority comparison [20]. We inspect the unmodified public `MemTXManager` at commit `4e1124cc`, the implementation registered as `memtx` in its experiment configuration [22]. Before execution, the protocol pins all 1,255 tracked upstream blobs, our adapter/checker, and the environment.

The 24 traces comprise eight read probes, eight old-writer probes and eight fresh-writer controls; positive read controls are included within the read traces.

In all eight read probes, A begins before B commits and has no staged writes, yet A's open transaction retrieves B's later record, absent from A's captured snapshot. The probes span four risk tiers and initially empty or unrelated-anchor states. A fresh transaction after B includes and retrieves B as expected. Source inspection shows that retrieval filters the current store without filtering to the captured transaction IDs. This behavior is compatible with a latest-readable-state API; it differs from the separately described start-time read contract.

For old-writer probes, A stages an older observation of a single-valued synthetic door state; B commits a contradictory later observation before A commits. With an unrelated anchor making A's initial snapshot nonempty, all four source/authority variants reject A by the temporal rule. With an empty initial snapshot, equal-authority/same-source and higher-A-authority variants instead commit A and supersede B: two overwrites among eight old-writer cases. The other two empty cases reject by ordinary authority or review rules. The source's stale-rival check is guarded by snapshot nonemptiness. Eight controls in which A actually begins after B capture B and exercise intended authority-policy outcomes.

These are bounded paper/code conformance discrepancies. Clarification by the original authors could determine whether they are implementation or documentation defects. They do not refute MemTX's separately stated action-gating and cascading-repair invariants, evaluate model quality, or introduce a new snapshot-isolation principle. No public-source patch is made. The full original test invocation encounters a missing optional `tau2` collection dependency; after excluding only that adapter and installing missing public dependencies, all 94 unchanged non-`tau2` tests pass. The failed full invocation and intermediate dependency errors remain reported; the full suite is not labelled passing. The repository supplies our traces, protocol and file hashes; the upstream implementation, whose pinned repository has no license file, is fetched separately.

8.2 Effect/receipt grouping across SQLite and Redis

A separate extension implements four authored contracts on SQLite 3.51.2 and Redis 7.2.17: split effect/receipt commits; a grouped prepared-new-write path; in-primitive receipt/type preflight with grouped writes; and a single application key/row envelope. All profiles share a durable caller-side binding of operation identity to literal body, target version and authority epoch. All recheck version and epoch at the effect. These shared protections are not credited to backend primitives. The four profiles bundle different validation placement and storage boundaries; they are not a one-factor ablation or best possible vendor implementations.

Thirteen histories across four profiles and two backends declare 104 cells. Ninety-six execute, while eight remain explicitly inapplicable because the tested fault boundary does not exist in that profile. The campaign uses 154 fresh request processes and ten actual client exits at declared barriers. A real receipt-storage failure after caller entry inspection leaves an effect without a receipt in the grouped Redis Lua path; explicit SQLite rollback removes its preceding effect. In-primitive preflight prevents this particular fault in both backends. This illustrates documented differences between isolated script execution and rollback semantics [38, 39, 43], not a Redis defect. A script that reads the invalid receipt hash before writing would already detect that fault. A separate valid-schema SQLite trigger error also demonstrates that preflight cannot predict every later failure.

All 96 executed cells have zero duplicate effects and zero stale-epoch effects. After revoke, eight receipt-hit retries acknowledge historical success while current visibility remains false at the new epoch; revocation holds target version fixed, so epoch checks remain discriminating. A split-path exit after effect commit but before receipt commit yields a stale-version conflict on literal retry, without silently rebasing. Explicit receipt garbage collection removes acknowledgement evidence by design in every profile; it is not an incidental failure rate. Historical success, current authority, and current visibility therefore require separate observations.

The checker reads all 48 final SQLite files and independently reduces Redis's native append-only stream from an empty base to all 72 final keys, checking each mutation and sixteen native multi-write groups. A separate graceful Redis restart preserves these keys. Twenty-one focused tests include twelve forged-evidence rejection checks. The dedicated study server, public image digest, version pins, commands and data are retained. No existing company service is accessed. Graceful restart is not server power loss; neither native grouping nor these bounded races prove arbitrary distributed durability or whole-plan atomicity.

9 Discussion and Validity

The evidence distinguishes record address, chosen operation, historical effect, and current fact. Stable addresses repair duplicate physical insertion only under a stable binding between identity and operation. Retained choices preserve a decision, including an incorrect decision. Receipts suppress repeated work or recognize historical success; they do not keep current memory equal to an earlier intent's desired state forever.

The strongest additional observation is that exact-target presence can stay perfect while duplicate records reduce the number of distinct strings returned in the same number of slots. A candidate window adequate for selecting relevant context is not evidence of global absence for write admission. This is an executable interface obligation, not a new deduplication primitive. Complete-store checking here is a sequential intervention; safe concurrent admission and scalable uniqueness require further contracts and testing.

The primary counts are exact properties of authored cases, not estimates from a user population. The controlled Mem0 sweep deliberately chooses extraction behavior; the live campaign uses four batches and one requested alias/harness. Its prompt history is bounded and truncated. Retrieval queries are the stored baseline literals, selected states are post hoc, and no answer-generation workload or semantic grading is performed. Public-source discussion already describes related bounded-candidate risks. A larger number of cells, model calls or citations does not establish novelty by itself.

The MemTX probes check two stated snapshot boundaries and leave separately verified action/repair claims untouched. The Redis/SQLite profiles are authored application contracts with different validation placement; their observations do not rank vendor implementations. Explicit inapplicable cells, dependency failures, one ungraded live formulation, and negative controls remain visible. Same-author checkers, computational reproduction and hosted CI reduce implementation errors but do not replace outside replication.

Storage scope is local SQLite, Qdrant-client local mode, one framework checkpoint implementation, an in-process public memory implementation, and a dedicated local Redis server. Process exits follow declared semantic barriers rather than interrupting storage commits. Bounded races do not enumerate arbitrary distributed interleavings. Power loss, remote late commits, cluster consistency, arbitrary corruption, unbounded receipt retention and cross-user identity allocation remain untested. Visibility revocation is not physical erasure or model unlearning.

Further work should test realistic held-out workloads, additional memory implementations, independently adjudicated semantics and scalable concurrent admission. Independent replication and broader evidence of usefulness are needed to determine how far the observed mechanism transfers beyond these authored conditions.

10 Conclusion

A memory can preserve its literal fact set and return every queried target while accumulating physical copies that displace distinct retrieval context. In the tested public Mem0 path, a ten-candidate admission window explains that conjunction under explicit extraction conditions; native controlled calls, live process recovery and a fixed-reader exposure test make the mechanism inspectable. The factorial and additional public contract probes show why physical identity, decision continuity, historical completion, current visibility and backend failure behavior require separate checks. These bounded results motivate admission and recovery conformance alongside semantic memory evaluation.

References

[1] Peter Alvaro, Joshua Rosen, and Joseph M. Hellerstein. Lineage-driven Fault Injection. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 331–346. ACM, 2015. doi: 10.1145/2723372.2723711. URL https://people.ucsc.edu/~palvaro/molly.pdf.

[2] BAAI. bge-small-en-v1.5: Model card. Official model card, 2023. URL https://huggingface.co/BAAI/bge-small-en-v1.5. 384-dimensional English embedding model; accessed 2026-10-29.

[3] Deniz Bilgin, Jonas Schewior, and Andreas Wendemuth. Capture is easy, forgetting is hard: Benchmarking prospective memory in conversational agents. In *Proceedings of the Second Workshop for REsearch on Agent Language Models (REALM 2026)*, 2026. URL https://aclanthology.org/2026.realm-1.43/.

[4] Andrew D. Birrell and Bruce Jay Nelson. Implementing Remote Procedure Calls. *ACM Transactions on Computer Systems*, 2(1):39–59, 1984. doi: 10.1145/2080.357392. URL https://www.birrell.org/andrew/papers/.

[5] Edward Y. Chang and Longling Geng. Sagallm: Context management, validation, and transaction guarantees for multi-agent llm planning. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2503.11951.

[6] Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchi Li, Feiyu Xiong, and Zhiyu Li. Halumem: Evaluating hallucinations in memory systems of agents. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2511.03506.

[7] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *Advances in Neural Information Processing Systems 37*, pages 130185–130213. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-4136. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb113910e9c3f6242541c1652e30dfd6-Abstract-Conference.html.

[8] Zheng Chen, Hanqing Liu, Duling Xu, Dong Dong, Jialin Li, Bangzheng Pu, and Jidong Zhai. Cordon: Semantic Transactions for Tool-Using LLM Agents. arXiv preprint, 2026. URL https://arxiv.org/abs/2606.17573v1. Version 1; preprint.

[9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2504.19413.

[10] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen J. Xiang. Memory Injection Attacks on LLM Agents via Query-Only Interaction. In *Advances in Neural Information Processing Systems 38*, pages 52119–52153. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025. doi: 10.52202/085713-1554. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/42a97bbd9844d2bf68596730af80bcdf-Abstract-Conference.html.

[11] Malcolm Featonby. Making retries safe with idempotent APIs. The Amazon Builders' Library, n.d. URL https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/. Section: Same client request ID, different intent. Accessed October 29, 2026.

[12] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD international conference on Management of data - SIGMOD '87*, pages 249–259. ACM Press, 1987. doi: 10.1145/38713.38742. URL https://www.cs.princeton.edu/research/techreps/598. Primary text inspected in Princeton technical report TR-070-87, January 1987.

[13] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2507.05257.

[14] jianglong-nie. Mem0. MiniMem public code-reading note, 2026. URL https://github.com/jianglong-nie/MiniMem/blob/56da9878747104aefc29a51d178cff22d5bf6870/references/paper-reading/Mem0.md. Public repository commit dated 2026-07-30; source-inspection date stated inside the note differs; accessed 2026-10-29.

[15] Sajjad Khan. Verified Detection and Prevention of Concurrency Anomalies in Multi-Agent Large Language Model Systems. arXiv preprint, 2026. URL https://arxiv.org/abs/2606.17182v1. Version 1; preprint.

[16] LangChain. Graph API overview. LangGraph documentation, n.d. URL https://docs.langchain.com/oss/python/langgraph/graph-api. Section: Re-execution and idempotency. Accessed October 29, 2026.

[17] Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2512.12818.

[18] Collin Lee, Seo Jin Park, Ankita Kejriwal, Satoshi Matsushita, and John Ousterhout. Implementing Linearizability at Large Scale and Low Latency. In *Proceedings of the 25th Symposium on Operating Systems Principles*, pages 71–86. ACM, 2015. doi: 10.1145/2815400.2815416. URL https://seojinpark.net/projects/rifl/.

[19] Jiapeng Li. Where does exactly-once live? model, harness, and tool-contract effects on duplicate side effects in llm agents. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2609.29095.

[20] Xiaoyang Li, Yiqi Wang, Haohui Lu, Zhi Chen, Mo Li, Pingan Song, Mingkai Zheng, and Taotao Cai. MemTX: Transactional Belief Commit for Stateful Agent Memory. arXiv preprint, 2026. URL https://arxiv.org/abs/2607.23929v2. Version 2; preprint.

[21] Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, Junpeng Ren, Zehao Lin, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhiqiang Yin, Qingchen Yu, Bo Tang, Hongkang Yang, Zhi-Qin John Xu, and Feiyu Xiong. Memos: An operating system for memory-augmented generation (mag) in large language models. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2505.22101.

[22] lxy1134. MemTX—code and data. Public source repository, 2026. URL https://github.com/lxy1134/MEMTX_/tree/4e1124ccd5fd384857463020e3497aa73ec93be6. Commit 4e1124ccd5fd384857463020e3497aa73ec93be6; accessed 2026-10-29.

[23] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. doi: 10.18653/v1/2024.acl-long.747. URL https://aclanthology.org/2024.acl-long.747/.

[24] Isham Kalappurackal Mansoor, Abhishek Phadke, and Pratip Rana. Verified tool calls improve LLM agent reliability under non-atomic failures. *arXiv preprint arXiv:2608.02645*, 2026. doi: 10.48550/arXiv.2608.02645. URL https://arxiv.org/abs/2608.02645.

[25] Deekshith Marla and Kushagra Bhatnagar. Permission to retry: A study of authorization and recovery in ai agent workflows. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2610.31916.

[26] Mem0 contributors. Mem0: Python memory implementation. Public source repository, 2026. URL https://github.com/mem0ai/mem0/blob/26cdefb46600c0fa5953c4b5ee5003c125539fc4/mem0/memory/main.py. Python package mem0ai 2.2.1; commit 26cdefb46600c0fa5953c4b5ee5003c125539fc4; accessed 2026-10-29.

[27] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. Atomix: Timely, transactional tool use for reliable agentic workflows. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2602.14849.

[28] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. doi: 10.1145/128765.128770. URL https://doi.org/10.1145/128765.128770.

[29] OpenAI. Non-interactive mode. Codex documentation, n.d. URL https://learn.chatgpt.com/docs/non-interactive-mode. Accessed October 29, 2026.

[30] Abdelghny Orogat and Essam Mansour. Is Agent Memory a Database? Rethinking Data Foundations for Long-Term AI Agent Memory. arXiv preprint, 2026. URL https://arxiv.org/abs/2605.26252v1. Version 1; preprint.

[31] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint*, 2023. URL https://arxiv.org/abs/2310.08560.

[32] PhilOleson. Design: mention-and-recency-aware retrieval. Mem0 GitHub Discussion 6726, 2026. URL https://github.com/mem0ai/mem0/discussions/6726. Posted 2026-08-02; accessed 2026-10-29.

[33] Python Software Foundation. os._exit: process termination in Python 3.12, 2026. URL https://docs.python.org/3.12/library/os.html#os._exit. Official documentation; publication date unspecified; accessed 2026-10-29.

[34] Qdrant contributors. Qdrant python client. Software release, 2026. URL https://github.com/qdrant/qdrant-client/tree/v1.20.0. Version 1.20.0; local-mode client used in this study; accessed 2026-10-29.

[35] Qdrant contributors. FastEmbed. Software release, 2026. URL https://github.com/qdrant/fastembed/tree/v0.9.1. Version 0.9.1; accessed 2026-10-29.

[36] Peijun Qing, Fobo Shi, and Soroush Vosoughi. Utilmem: Benchmarking evidence utilization in long-term conversational memory. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, 2026. URL https://aclanthology.org/2026.emnlp-main.1639/.

[37] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2501.13956.

[38] Redis. Scripting with lua, 2026. URL https://redis.io/docs/latest/develop/programmability/eval-intro/. Official documentation; accessed 2026-10-29.

[39] Redis. Transactions, 2026. URL https://redis.io/docs/latest/develop/using-commands/transactions/. Official documentation; accessed 2026-10-29.

[40] Yi Ting Shen, Kentaroh Toyoda, and Alex Leung. Revoked but Still Authoritative: An Empirical Study of Revocation Enforcement in Agent-Memory Systems. arXiv preprint, 2026. URL https://arxiv.org/abs/2609.08258v1. Version 1; preprint.

[41] Yiwen Song, Yale Song, Tomas Pfister, and Jinsung Yoon. Paperorchestra: A multi-agent framework for automated ai research paper writing. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2604.05018.

[42] SQLite. Atomic Commit In SQLite, 2026. URL https://www.sqlite.org/atomiccommit.html. Official documentation; publication date unspecified; accessed 2026-10-29.

[43] SQLite. Transaction, 2026. URL https://www.sqlite.org/lang_transaction.html. Official documentation; accessed 2026-10-29.

[44] Zhaoyan Sun, Xiaoxiao Wang, and Guoliang Li. Agentic transaction: Towards acid-compliant agent systems. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2608.13900.

[45] Md Nayem Uddin, Kumar Shubham, Eduardo Blanco, Chitta Baral, and Gengyu Wang. From Recall to Forgetting: Benchmarking Long-Term Memory for Personalized Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 26814–26841. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.findings-acl.1337. URL https://aclanthology.org/2026.findings-acl.1337/.

[46] Ziming Wang. TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. arXiv preprint, 2026. URL https://arxiv.org/abs/2606.06240v1. Version 1; preprint.

[47] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2410.10813.

[48] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2502.12110.

[49] Chao Yao, Yangbo Wei, Zhen Huang, Junhong Qian, Chenle Chen, Shaoqiang Lu, Chen Wu, and Lei He. Forgetting Without Restarting: Execution-State Unlearning for Stateful LLM Agents. arXiv preprint, 2026. URL https://arxiv.org/abs/2609.04875v1. Version 1; preprint.

[50] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. Fault-tolerant and transactional stateful serverless workflows. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1187–1204. USENIX Association, 2020. URL https://www.usenix.org/conference/osdi20/presentation/zhang-haoran.

[51] Yusheng Zheng, Yiwei Yang, Wei Zhang, and Andi Quinn. Acrfence: Preventing semantic rollback attacks in agent checkpoint-restore. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2603.20625.

[52] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *arXiv preprint*, 2023. URL https://arxiv.org/abs/2305.10250.

[53] Fangxing Zhou, Yifei Sun, Jiachen Li, Jinyu Xu, Yuhan Wang, Ruixin Li, and Qing Xie. Timlmembench: A diagnostic benchmark for governed use of long-term memory in dialogue agents. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. URL https://aclanthology.org/2026.findings-emnlp.507/.

[54] Genliang Zhu and Chu Wang. Authorization Revocation for Long-Running AI Agents: Root-Scoped Quiescence under Delegation and Asynchronous Execution. arXiv preprint, 2026. URL https://arxiv.org/abs/2609.21284v1. Version 1; preprint.

A Artifacts, Reproduction, and Assistance

The public companion repository is `https://github.com/aditya-genies/beyond-recall`. It contains synthetic inputs, frozen primary data, databases, checkers, figure sources, extension protocols, model prompts/responses, and dependency pins. Source/data manifests identify each campaign separately; counts are not pooled.

```
python3 -W error::ResourceWarning -m unittest -v test_study
python3 study.py campaign --out my-results --workers 4
python3 checks/check_results.py --campaign my-results \
  --out my-verification.json
python3 compare_results.py --candidate my-results
python3 extensions/sol_pilot/check.py \
  --out extensions/sol_pilot/registered
python3 extensions/conflict_contracts/check.py \
  --out extensions/conflict_contracts/results
```

Native-framework and integration instructions are in their extension directories. The model runners require an authenticated personal Codex account and have no API-key fallback. New Mem0, MemTX and backend instructions, frozen specifications, checker tests and separate source pins are in `extensions/replanning_contracts/`. Its model/archive verification does not require fresh model calls. Mem0 model weights and unlicensed upstream MemTX implementation are fetched separately; neither is redistributed. A fresh model request need not reproduce its literal output; archived responses make downstream integration rerunnable without model access.

The primary source digest remains

`98378f217f0dae050dbc07aa04e1121ed9bc73862c2ee35a2e31a7a464f4e9b9`

The primary cases digest remains

`6714681fbefcbdf63334f76b95410aa1c2bc620ce8b7511fadb0c91354b72d01`

Full SHA-256 manifests cover remaining evidence. New experiments have separate specifications and do not overwrite the primary study.

PaperOrchestra guided outline, literature, figures, drafting, and refinement [41]. AI assistance also supported implementation, tests, checks, and prose. The labelled Sol 6.1 pilot and native-ingestion requests are experimental data, separate from authoring assistance. The author is responsible for claims and release decisions. This is a working manuscript; public availability is not peer-reviewed publication or an acceptance claim.